RIPHAH INTERNATIONAL UNIVERSITY ISLAMABAD

Faculty of Computing, BSSE (6-1), Spring 2026

Semester Project – Software Design & Architecture

Team Members:

Sap ID:	**Name:**
54481	Hassan Zahid
54523	Malik Hamza Shahid

SDA-Pro

Security Incident Response & Threat Mitigation Platform

Software Design & Architecture — Semester Project

GitHub repository Link**GitHub Repository Link:** https://github.com/Hassan-Zahid-07/SDA-Pro

Field	**Details**
Project Type	Java and React based SDA semester project with working prototype and dashboard evidence
Core Domain	Security Operations Center (SOC), alert triage, threat enrichment, incident response
Implementation Style	Plain Java core implementation with a React-based SOC web dashboard and documented architecture evidence
Architecture Styles	SOA, MVC, Layered Architecture, Event-Driven Architecture
Design Patterns	All 12 required design patterns implemented and annotated in code

Prepared as a complete academic deliverable with source code, UML, ADRs, API documentation, testing evidence, and Git-ready repository structure.

Table of Contents

1. Executive Summary

SDA-Pro is a Security Operations Center (SOC) prototype designed to demonstrate practical software design and architecture concepts in a realistic cybersecurity domain. The system ingests alerts from multiple security sources, normalizes them into a canonical model, enriches them through a configurable processing pipeline, creates incidents, executes response actions, and publishes real-time updates to observers such as the Java dashboard, audit logger, notification dispatcher, metrics collector, and the React-based SOC web dashboard.

The core backend prototype is implemented in plain Java so the focus remains on architecture, class responsibilities, object collaboration, and design pattern usage instead of framework configuration. A separate React-based SOC dashboard module has been added under soc-dashboard/ to provide a professional GUI, demonstrate MVC separation, and visually present alert streams, incident queues, lifecycle states, response actions, threat intelligence results, and audit events.

Assessment Area	**Current Position**
Implementation	Working Java prototype with mock connectors, in-memory repositories, end-to-end demo flow, and React SOC dashboard prototype
Documentation	README, ADRs, API contracts, UML diagrams, evidence folders, report, and presentation
Architecture	Clear separation into service packages representing SOA, Java MVC evidence, React MVC dashboard, layered, and event-driven design
Testing	Manual run evidence and smoke test output available through demo-output.txt and terminal screenshots
Git Repository	Repository contains .gitignore, documentation structure, and clean files.

SDA-PRO SOC COMMAND CENTER

Security Incident Response & Threat Mitigation Platform

Real-time dashboard prototype for alert monitoring, incident triage, automated response, threat intelligence, and audit visibility.

Active Analyst
Hassan Zahid
SOC Manager · Online

Total Alerts	Critical Incidents	Actions Executed	Threat Intel Matches
128	06	24	17

Real-Time Alert Stream

Ingested alerts from SIEM, firewall, EDR, and threat intelligence sources.

Simulate Alert

ALERT ID	SOURCE	TYPE	SEVERITY	IP	STATUS	TIME
SENT-ALERT-SIEM-204	Splunk SIEM	Suspicious Login	HIGH	203.0.113.45	Enriched	10:41 AM
FW-204	Palo Alto Firewall	Command & Control Traffic	CRITICAL	203.0.113.45	Incident Created	10:42 AM
EDR-204	CrowdStrike EDR	Endpoint Malware Signal	HIGH	203.0.113.45	Correlated	10:43 AM

Incident Queue

Prioritized incident list for SOC analysts.

INC-001
Multi-Vector APT Campaign
CRITICAL
Containment

INC-002
Suspicious Privilege Escalation
HIGH
Under Triage

Incident Lifecycle Tracker

State Pattern evidence for INC-001: Multi-Vector APT Campaign

Advance State

1 New · 2 Under Triage · 3 Containment · 4 Eradication · 5 Recovery · 6 Post-Incident Review · 7 Closed

Asset	Analyst	Strategy	Risk Score
Finance-Workstation-22	SOC Manager	Aggressive Containment	92/100

Response Actions

Facade coordinates Strategy, Factory, Decorator, and Proxy execution.

Block IP
203.0.113.45
Factory Method + Proxy + Decorator

Isolate Endpoint
Finance-Workstation-22
Strategy + Facade + Decorator

Quarantine File
payload.exe
Factory Method + Rollback Decorator

Escalate to Tier-3
Tier-3 Analyst
Strategy + Notification Factory

Threat Intelligence

Adapter + Proxy evidence for external intelligence providers.

92
Malicious Reputation Score

Indicator: 203.0.113.45
Provider: VirusTotal + MISP
Cache: Hit through CachingThreatIntelProxy
Verdict: Command-and-control infrastructure

Audit Log & Observer Events

Immutable-style event evidence from the simulated EventBusPublisher.

```
[10:41 AM] Alert ingested from Splunk SIEM
[10:42 AM] Alert ingested from Palo Alto Firewall
[10:43 AM] Alert ingested from CrowdStrike EDR
[10:44 AM] Chain of Responsibility enrichment completed
[10:45 AM] Incident INC-001 created and published through EventBusPublisher
[10:46 AM] DashboardUpdater received Observer notification
```

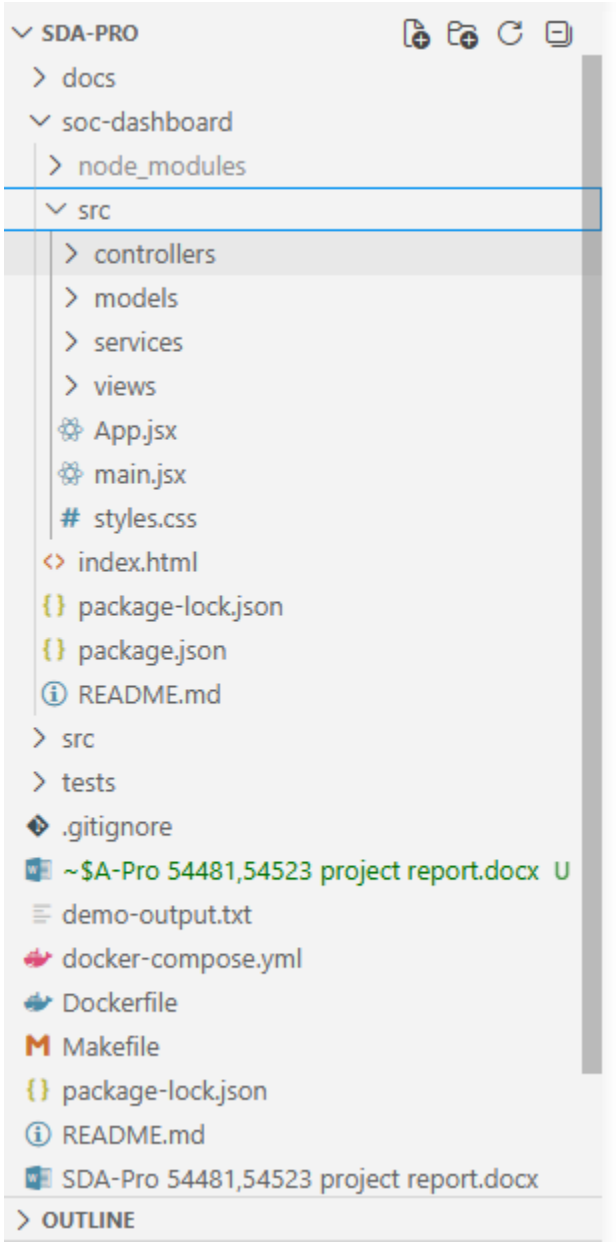
2. Project Requirements and Compliance

The project was evaluated against the three main submission expectations given for the semester project: proper implementation, well-structured documentation, and effective version control management through Git repositories.

- All required design patterns are present in the code and are annotated with clear pattern comments.
- The demo output verifies multiple alert sources and all major processing stages.
- The documentation folders provide traceability from requirements to code and diagrams.

3. Repository Structure and Documentation Evidence

The repository is organized so that source code, documentation, evidence, test output, and submission artifacts are easy to locate. This improves readability for the evaluator and supports the requirement of well-structured documentation.



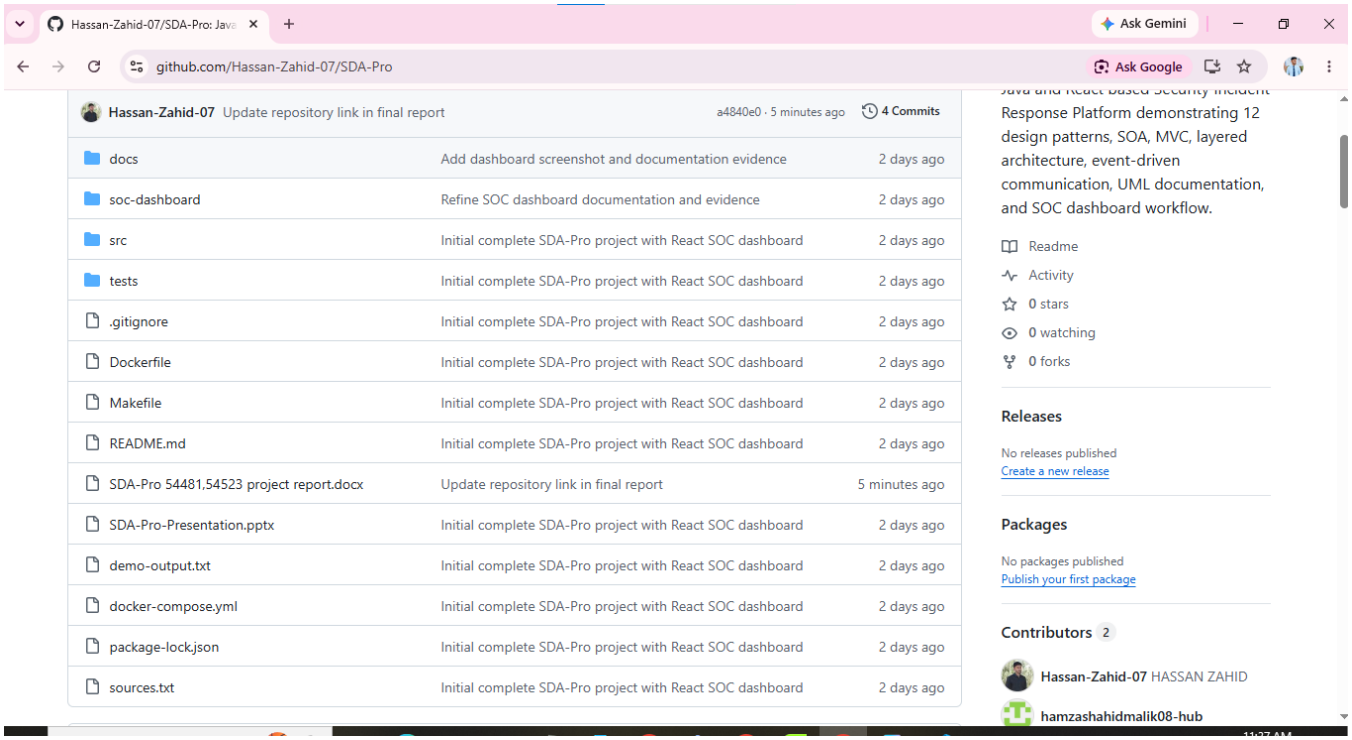


Figure 1: Project repository structure showing docs, screenshots, source packages, tests, Docker files, Makefile, README, report, and presentation.

3.1 Documentation Folder Organization

The documentation folder is divided into ADRs, API contracts, evidence, screenshots, and UML artifacts. This layout makes the project easy to review because every architectural claim has a supporting document or screenshot.

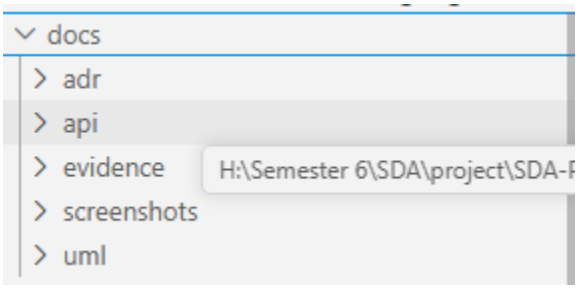


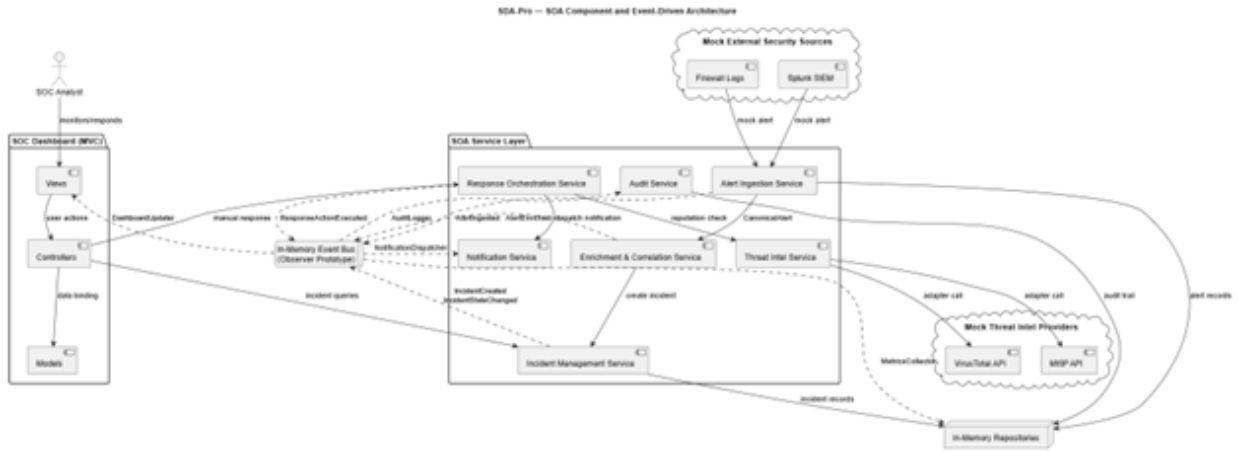
Figure 2: Documentation folder containing adr, api, evidence, screenshots, and uml subfolders.

Folder	Purpose
docs/adr	Architecture Decision Records explaining major design choices
docs/api	Service contracts and API-level communication evidence
docs/uml	Class, component, and sequence diagrams
docs/evidence	Supporting proof for implementation and submission
docs/screenshots	Visual execution and structure screenshots used in the final report

4. System Architecture

SDA-Pro applies four required architecture styles: Service-Oriented Architecture, MVC, Layered Architecture, and Event-Driven Architecture. These styles are not only documented in diagrams; they are reflected in the package structure and runtime flow of the Java project.

Architecture Style	Application in SDA-Pro	Evidence
SOA	Core capabilities are separated as service-oriented packages such as ingestion, enrichment, incident, response, threat intelligence, notification, audit, and dashboard.	src package structure and component diagram
MVC	The project contains Java MVC dashboard evidence and a separate React web dashboard organized into controllers, models, views, and services.	src/dashboard/controllers, models, views; soc-dashboard/src/controllers, models, views, services
Layered Architecture	Domain objects, service logic, adapters, events, and repositories are separated by responsibility.	package hierarchy and source folders
Event-Driven Architecture	Domain events are published by EventBusPublisher and consumed by dashboard, audit, notification, and metrics observers.	events package and demo output



4.1 SOC Analyst Web Dashboard (React MVC Prototype)

To make the SOC monitoring workflow more intuitive and presentation-ready, a React-based web dashboard was added under the soc-dashboard/ directory. This dashboard does not replace the Java implementation; it complements it by providing a professional user interface for the same SOC workflow. It visualizes total alerts, critical incidents, executed actions, threat-intelligence matches, real-time alert streams, incident queue, incident lifecycle tracker, response actions, threat intelligence results, and audit/observer events.

The dashboard follows a clear MVC-inspired structure. Controllers manage user actions and simulated state transitions, models represent alert, incident, response action, and audit data, views render the dashboard interface, and services simulate event-driven updates through a mock event bus.

The implementation evidence is organized as follows:

- Controller layer: soc-dashboard/src/controllers/DashboardController.js handles simulated analyst actions and state changes.
- Model layer: soc-dashboard/src/models/*.js stores alert, incident, response action, and audit log data.
- View layer: soc-dashboard/src/views/*.jsx renders overview cards, alert stream, incident queue, lifecycle tracker, response actions, threat intelligence, and audit logs.
- Service layer: soc-dashboard/src/services/mockEventBus.js simulates Observer/Event-Driven updates without affecting the Java backend.
- Application entry and styling: App.jsx, main.jsx, and styles.css compose the interface and provide a clean SOC command-center design.

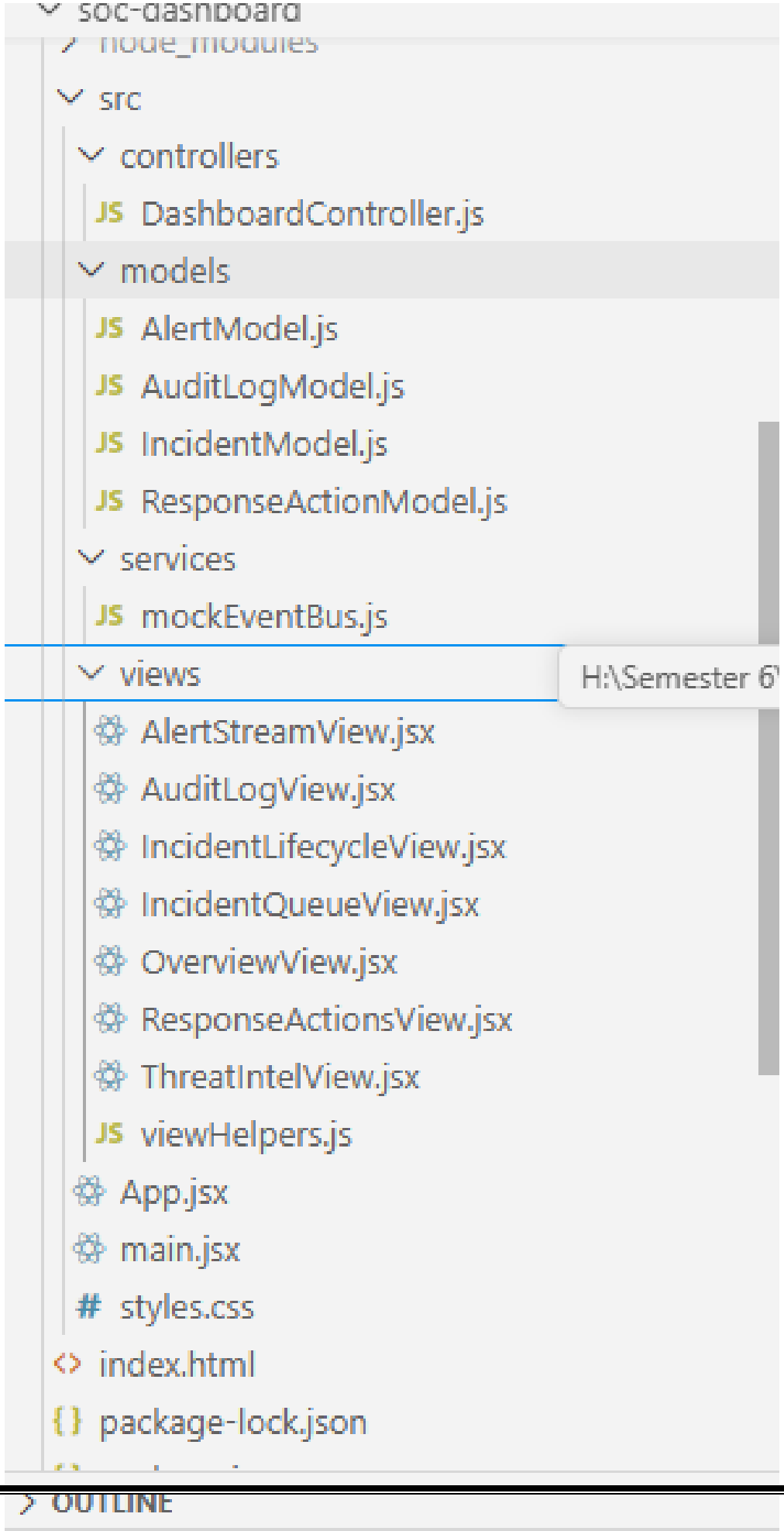

soc-dashboard
node_modules
src
controllers
DashboardController.js
models
AlertModel.js
AuditLogModel.js
IncidentModel.js
ResponseActionModel.js
services
mockEventBus.js
views
H:\Semester 6\
AlertStreamView.jsx
AuditLogView.jsx
IncidentLifecycleView.jsx
IncidentQueueView.jsx
OverviewView.jsx
ResponseActionsView.jsx
ThreatIntelView.jsx
viewHelpers.js
App.jsx
main.jsx
styles.css
index.html
package-lock.json
OUTLINE

Figure 3A: React SOC dashboard source structure showing controllers, models, services, views, App.jsx, main.jsx, and styles.css.

Figure 3: Component diagram showing SOA services, dashboard, event-driven communication, mock external sources, and in-memory repositories.

5. Design Pattern Implementation

The project implements all 12 required design patterns. Each pattern was applied because it solves a specific domain problem in the SOC workflow. This avoids forced pattern usage and makes the design easier to explain during presentation or viva.

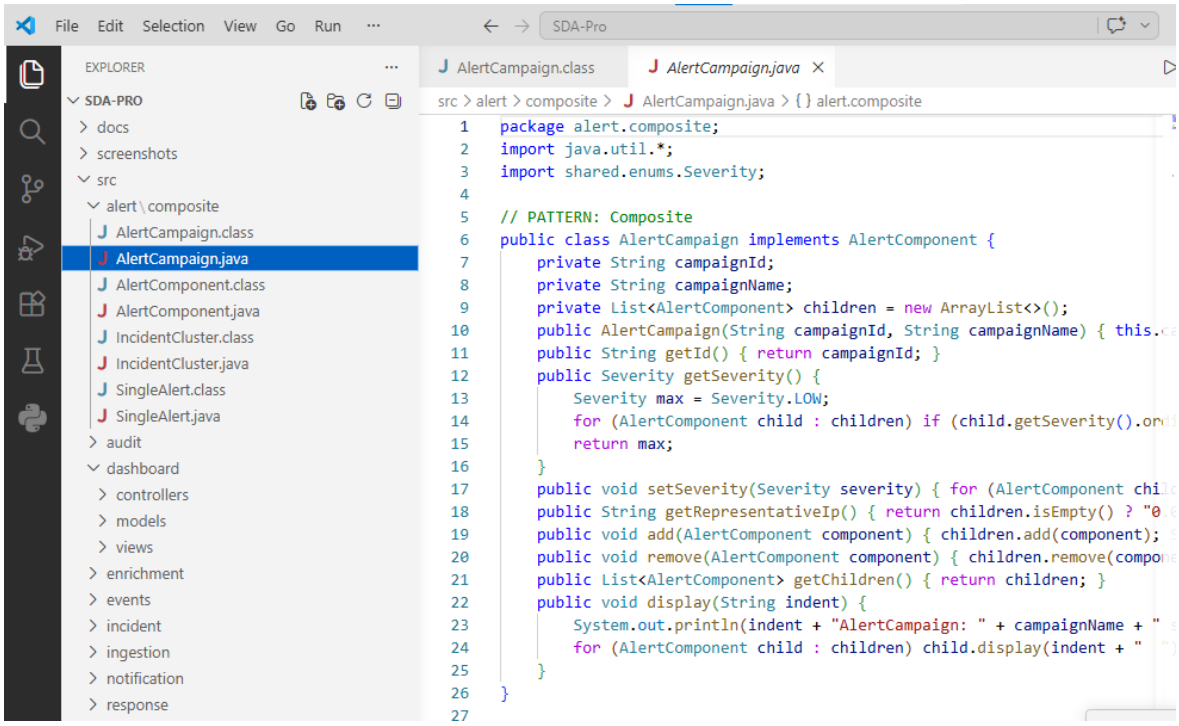


Figure 4: Code evidence showing the Composite pattern annotation in AlertCampaign.java and package organization around implemented patterns.

#	Pattern	Main Evidence	Reason for Use
1	Singleton	IngestionConfigManager, ThreatIntelCache, EventBusPublisher	Shared configuration, cache, and event publisher
2	Factory Method	AlertNormalizerFactory, ResponseActionFactory	Create correct normalizer/action at runtime
3	Abstract Factory	EnrichmentProviderFactory, NotificationFactory	Create related families of providers/notifiers
4	Composite	AlertComponent, SingleAlert, AlertCampaign, IncidentCluster	Treat single alerts and grouped campaigns uniformly
5	Facade	IncidentResponseFacade	Hide response orchestration complexity from callers
6	Adapter	SplunkAdapter, FirewallAdapter, CrowdStrikeAdapter,	Convert external formats into project interfaces

		VirusTotalAdapter, MISPAdapter	
7	Decorator	AuditLogDecorator, ApprovalGateDecorator, RollbackDecorator, MetricsDecorator	Add behavior around response actions dynamically
8	Proxy	Caching, RateLimit, AccessControl, AuthorizedResponseActionProxy	Control expensive calls and authorization
9	State	NewState through ClosedState	Enforce legal incident lifecycle transitions
10	Chain of Responsibility	Deduplication, GeoIP, ThreatIntel, AssetContext, Classification	Process alerts through independent enrichment stages
11	Observer	DashboardUpdater, AuditLogger, NotificationDispatcher, MetricsCollector	Keep subscribers loosely coupled to publishers
12	Strategy	Aggressive, Balanced, Conservative, WatchAndWait	Choose response behavior based on severity and context

6. End-to-End System Flow

The running demo follows the main SOC scenario from external alert collection to automated response. The flow proves that the design patterns are collaborating in a meaningful workflow rather than existing as isolated classes.

1. Alert ingestion begins through mock external adapters for Splunk SIEM, Palo Alto Firewall, and CrowdStrike EDR.
2. AlertNormalizerFactory converts each source-specific alert into a canonical alert model.
3. Composite grouping places alerts into AlertCampaign or IncidentCluster structures.
4. The enrichment pipeline applies deduplication, GeoIP, threat intelligence, asset context, and classification stages.
5. IncidentManagementService creates and transitions incidents through the defined state lifecycle.
6. IncidentResponseFacade coordinates strategy selection, action creation, decorators, proxies, and event publishing.
7. EventBusPublisher notifies dashboard, audit, notification, and metrics subscribers.

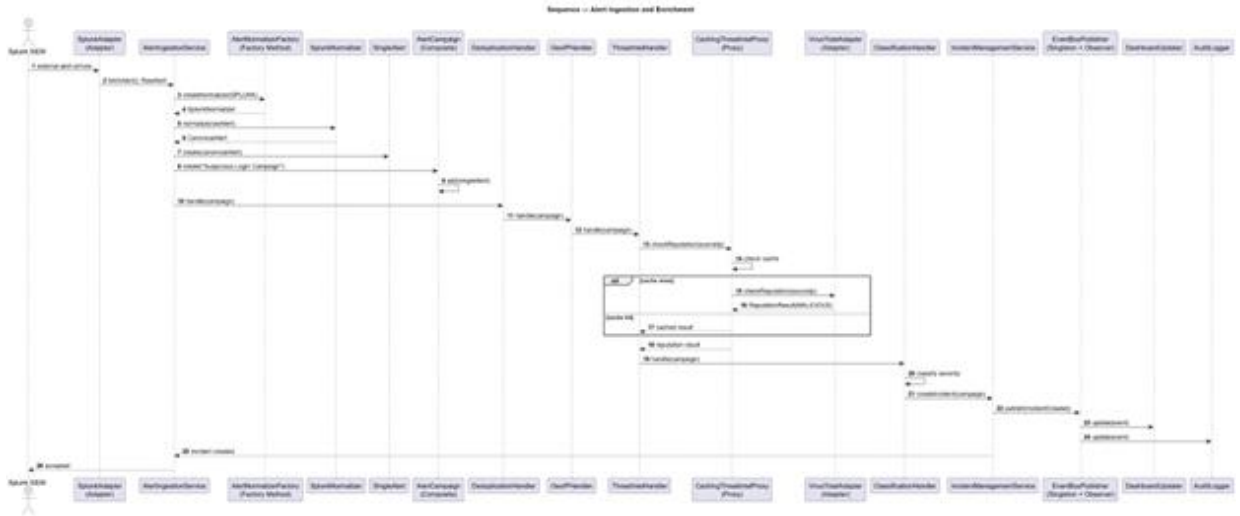


Figure 5: Sequence diagram for alert ingestion, normalization, enrichment, correlation, and incident creation.

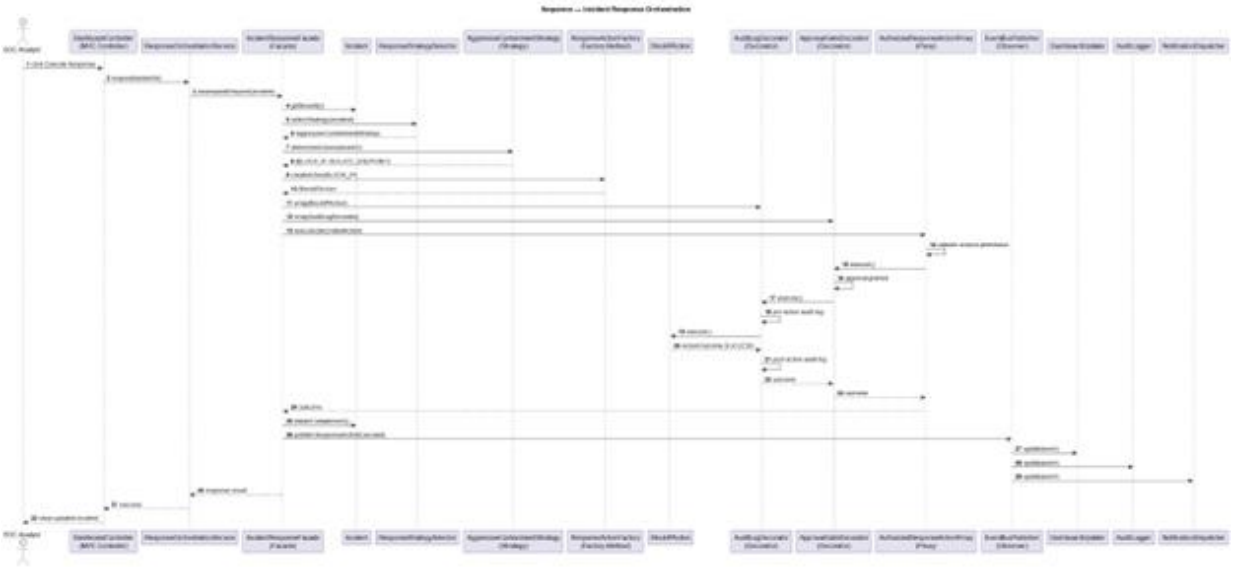


Figure 6: Sequence diagram for incident response orchestration using Facade, Strategy, Decorator, Proxy, State, and Observer.

7. Testing and Execution Evidence

The project was compiled and executed successfully. The demo output confirms that all major steps run in sequence, including three-source ingestion, enrichment, observer notifications, incident creation, and automated response actions.

```
OUTPUT ···   Filter (e.g. text, !excludeText, t...   Code

[Running] cd "h:\Semester 6\SDA\project\SDA-Pro\src\" && javac MainDemo.java && java
MainDemo
============================================================
  SDA-Pro -- Security Incident Response & Threat Mitigation Platform
  Software Design & Architecture -- Semester Project
  Team: Hassan Zahid (54481) | Malik Hamza Shahid (54523)
============================================================

--- STEP 1: Alert Ingestion from 3 Sources (Adapter + Factory Method) ---
[Adapter] Pulling SIEM alert from SDA-Pro Splunk mock connector...
[Factory Method] SIEM normalizer mapped raw login data to CanonicalAlert.
[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from Splunk SIEM
[DashboardUpdater] Live dashboard refresh: Alert ingested from Splunk SIEM
[Adapter] Pulling perimeter alert from SDA-Pro firewall mock connector...
[Factory Method] Firewall normalizer mapped network event to CanonicalAlert.
[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from Palo Alto Firewall
[DashboardUpdater] Live dashboard refresh: Alert ingested from Palo Alto Firewall
[Adapter] Pulling EDR alert from SDA-Pro CrowdStrike mock connector...
[Factory Method] CrowdStrikeNormalizer converted raw alert to CanonicalAlert.
[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from CrowdStrike EDR
[DashboardUpdater] Live dashboard refresh: Alert ingested from CrowdStrike EDR

--- STEP 2: Composite Grouping -- Campaign + IncidentCluster ---
[Composite] Added alert to campaign: Multi-Vector APT Campaign: SDA-Pro
[Composite] Added alert to campaign: Multi-Vector APT Campaign: SDA-Pro
[Composite] Added alert to campaign: Multi-Vector APT Campaign: SDA-Pro
  AlertCampaign: Multi-Vector APT Campaign: SDA-Pro severity=HIGH
    SingleAlert: CanonicalAlert{SENT-ALERT-SIEM-204, SPLUNK, severity=
    113.45}
                                                  Opening Java Projects
```

Figure 7: Terminal execution evidence showing the SDA-Pro demo starting and processing alerts from three sources.

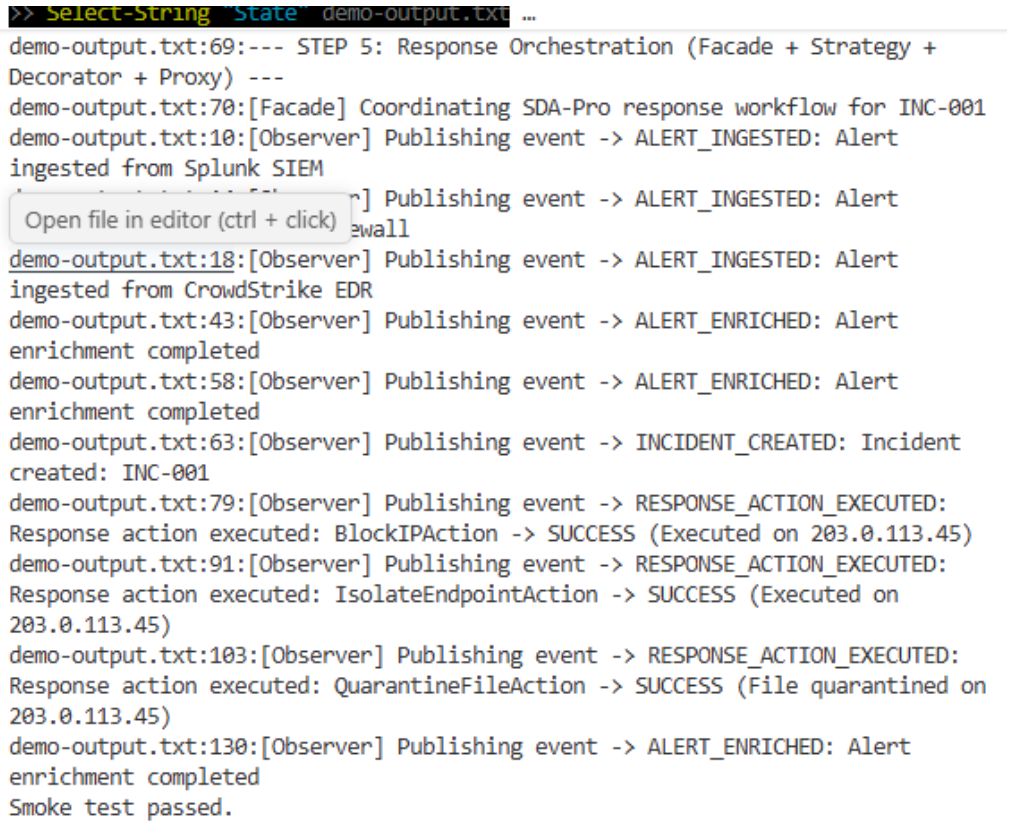


Figure 8: Smoke test evidence showing required pattern keywords found in demo-output.txt and final "Smoke test passed" message.

7.1 Commands Used for Local Verification

Verification Step	PowerShell Command
Compile Java source files	Get-ChildItem -Recurse src -Filter *.java | ForEach-Object { $_.FullName } > sources.txt javac -d out @sources.txt
Run main demo	java -cp out MainDemo | Tee-Object -FilePath demo-output.txt
Smoke test checks	Select-String "Adapter" demo-output.txt Select-String "Factory Method" demo-output.txt Select-String "Composite" demo-output.txt Select-String "Chain" demo-output.txt Select-String "State" demo-output.txt Select-String "Facade" demo-output.txt Select-String "Observer" demo-output.txt Write-Output "Smoke test passed."

8. Version Control Readiness

The project was prepared for GitHub-based version control and collaboration. The official repository is hosted under the Hassan-Zahid-07 account and includes the complete source code, documentation, UML artifacts, screenshots, report, presentation, Docker configuration, Makefile, Java backend, and React SOC dashboard. The second team member was added as a collaborator so both members can contribute through separate commits.

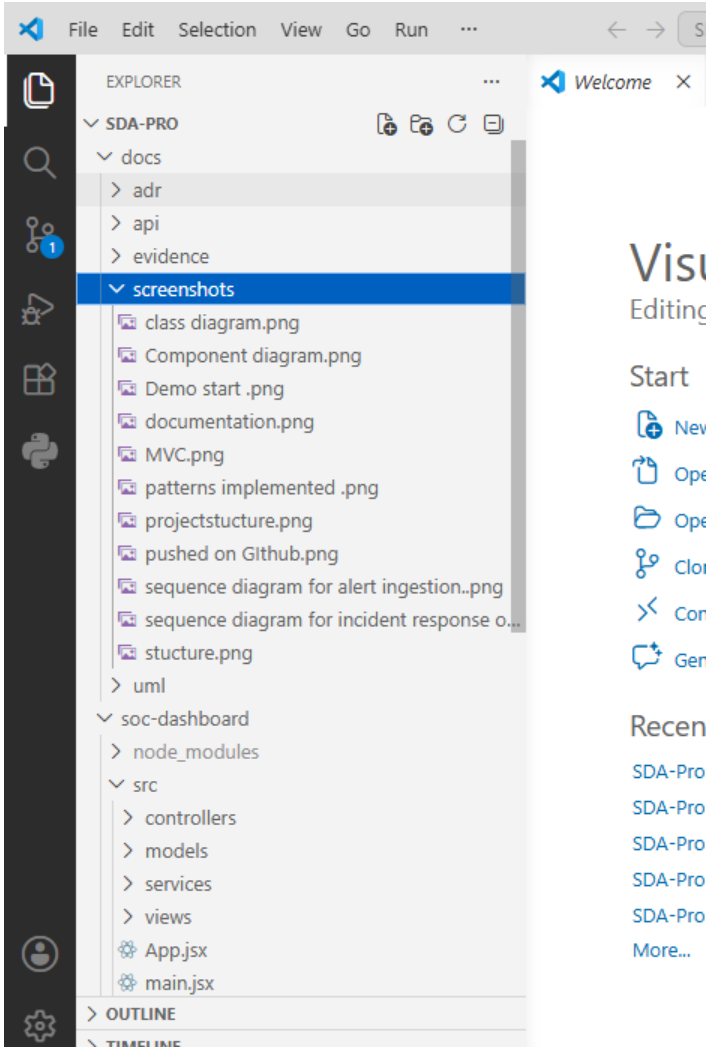
- Submission readiness: clean repository structure with source code, documentation, dashboard, test evidence, report, and presentation.
- Collaboration evidence: commit history and contributor list show project updates from team accounts.
- Branch used: main branch for final submission, with feature branches available for future improvements.
- Repository access: private repository with collaborator access for team contribution and academic review.
- Repository URL: https://github.com/Hassan-Zahid-07/SDA-Pro

9. Acceptance Criteria Matrix

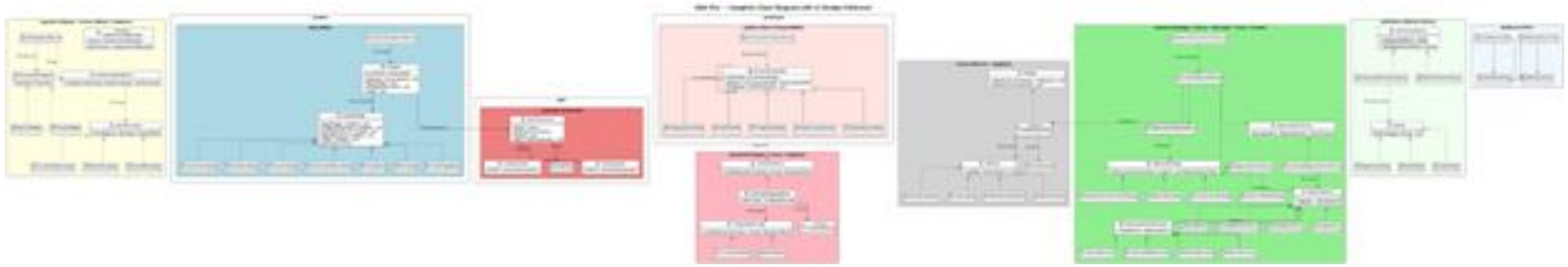
Acceptance Criterion	Evidence in Project	Status
Alerts from 2+ sources ingested and normalized	Splunk SIEM, Palo Alto Firewall, CrowdStrike EDR shown in demo output	Pass
Chain of Responsibility pipeline with at least 3 handlers	Deduplication, GeoIP, ThreatIntel, AssetContext, Classification	Pass
Composite grouping into campaigns/clusters	AlertCampaign and IncidentCluster classes	Pass
Incident lifecycle states	New, UnderTriage, Containment, Eradication, Recovery, PostIncidentReview, Closed	Pass
At least 2 response strategies	Aggressive, Balanced, Conservative, WatchAndWait	Pass
2+ threat intel adapters through proxy	VirusTotal and MISP adapters with caching/rate-limit/access-control proxies	Pass
Dashboard updates through Observer/Event-Driven pattern	DashboardUpdater receives EventBusPublisher notifications; React SOC dashboard visualizes alert stream, incident queue, lifecycle, response actions, threat intel, and audit events	Pass
All 12 patterns annotated in code	// PATTERN comments visible in source files	Pass
Architecture documented	UML diagrams, ADRs, README, API docs, dashboard evidence, and repository screenshots	Pass
System compiles and runs	Terminal demo and smoke test screenshots	Pass

Appendix A: Visual Evidence and Screenshots

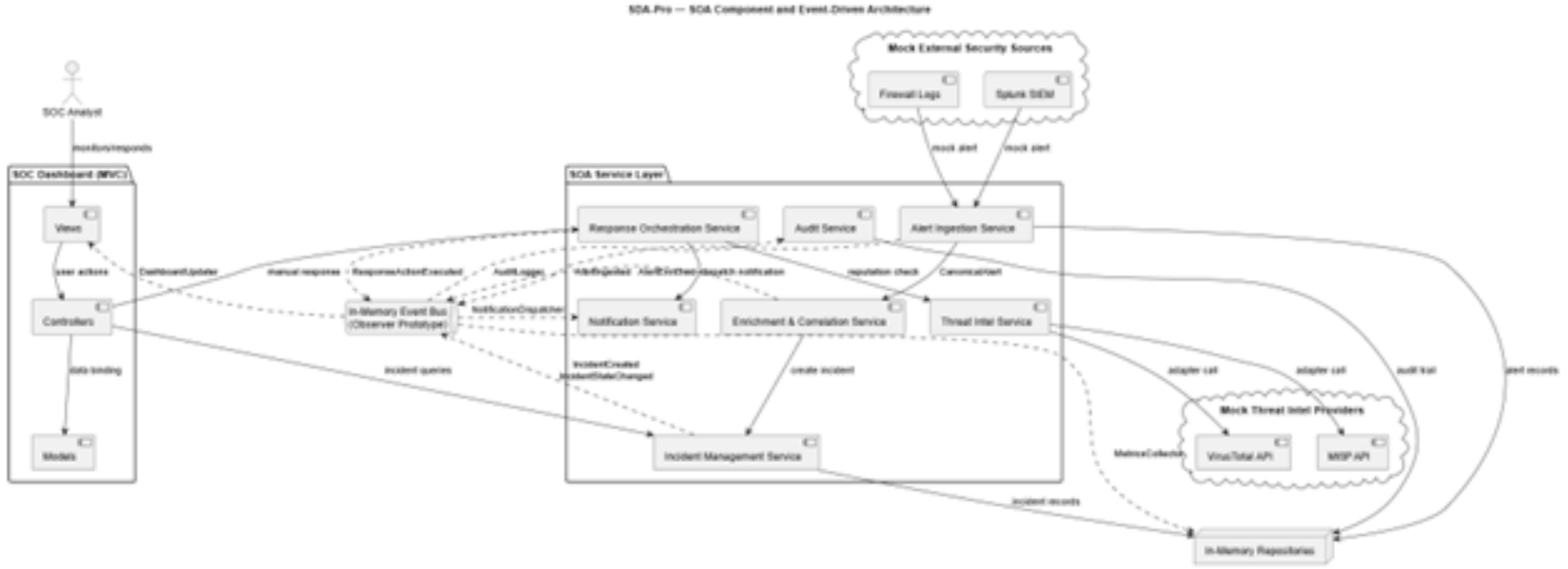
This appendix contains additional visual evidence added to make the report easier to verify. The screenshots show the repository structure, documentation, UML diagrams, implementation evidence, and execution output.



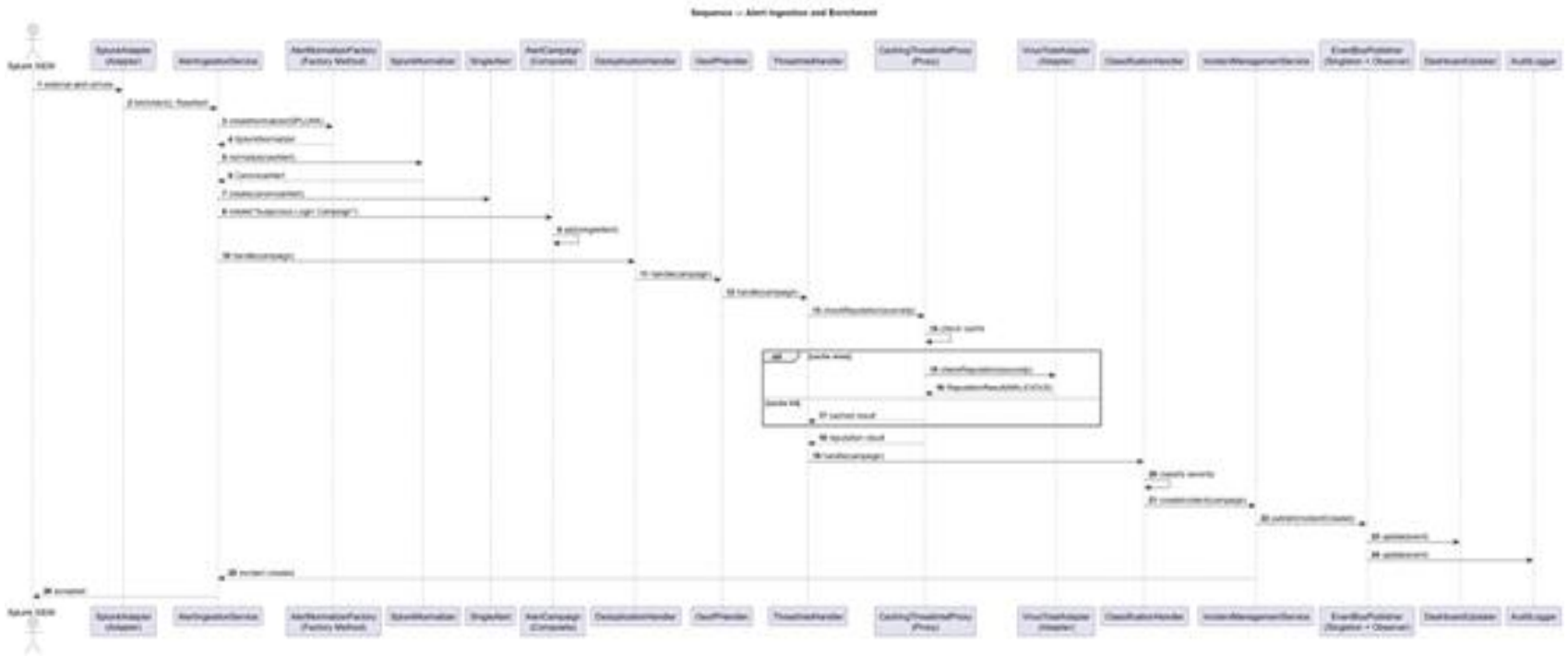
Appendix Figure A1: High-level docs folder structure from VS Code.



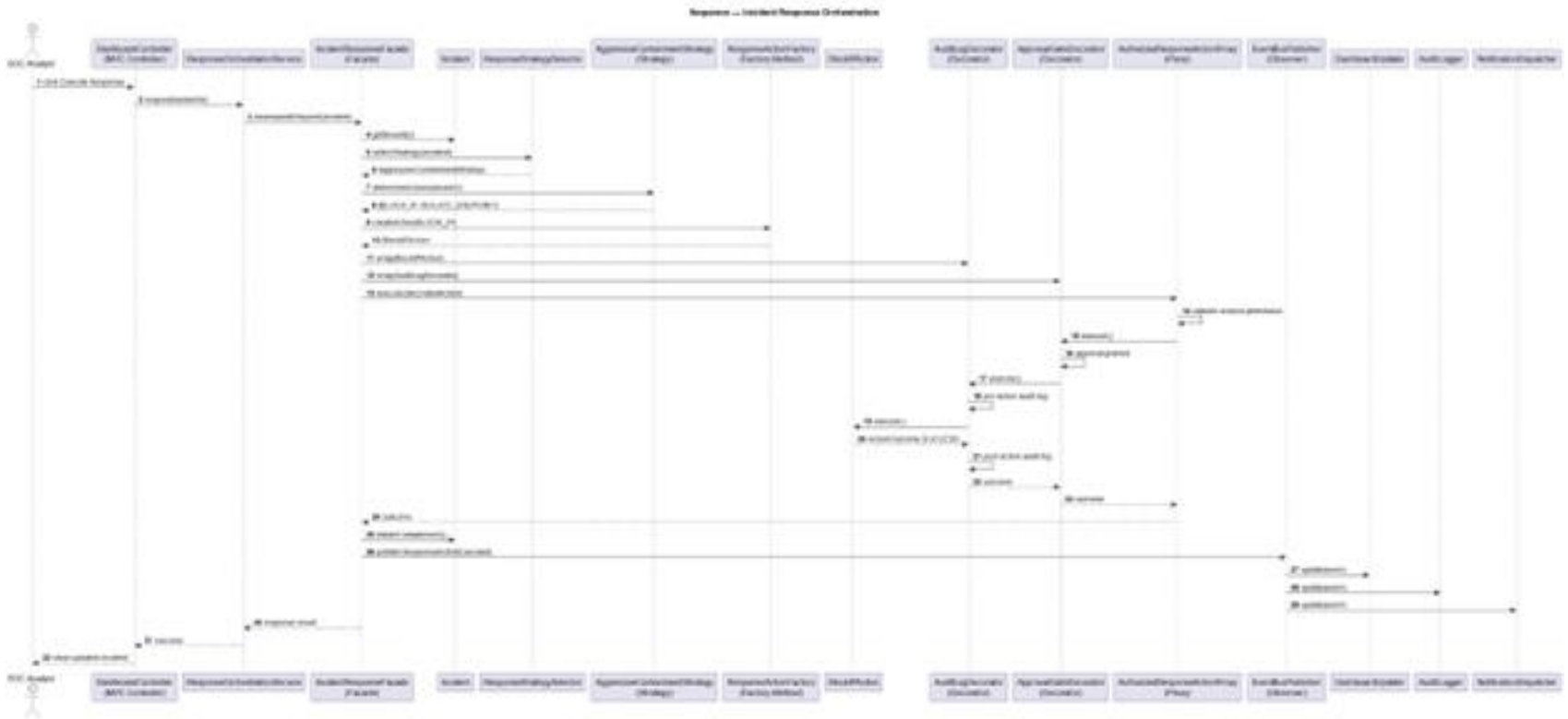
Appendix Figure A2: UML class diagram covering all required patterns.



Appendix Figure A3: SOA component and event-driven architecture diagram.



Appendix Figure A4: Alert ingestion and enrichment sequence diagram.



Appendix Figure A5: Incident response orchestration sequence diagram.